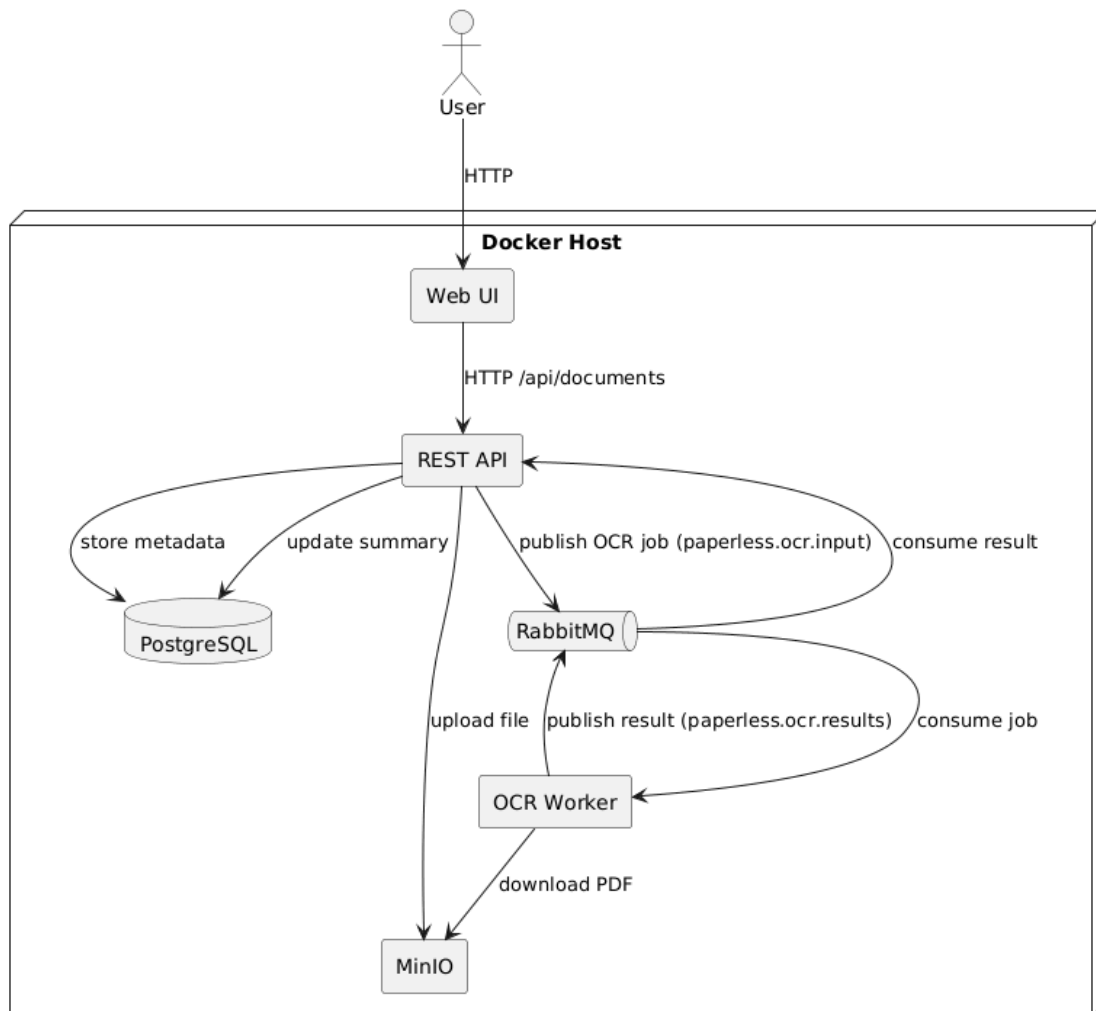


Architecture (Sprints 1–3)

1. Components (as of Sprint 3):

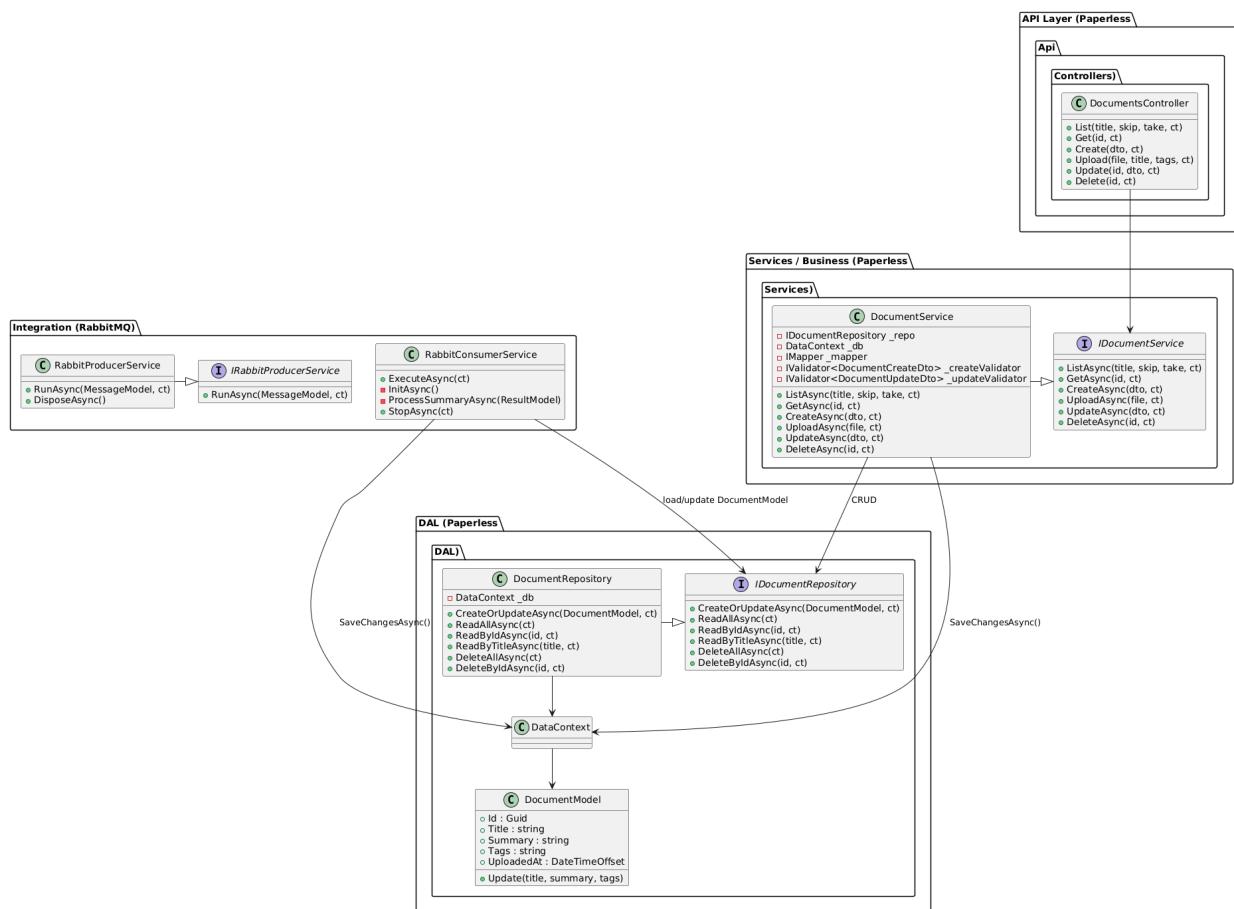
- Frontend (Quasar app in frontend):
 - Runs in container paperless-frontend behind nginx web server
- REST API (paperless backend):
 - Handles document upload and retrieval
 - Runs in container paperless-backend
- PostgreSQL:
 - Persists document metadata and other entities
 - Runs in container paperless-postgres
- RabbitMQ:
 - Message broker for document processing
 - Runs in container paperless-rabbitmq
- OCR-worker
 - Runs in container paperless-ocrworker
- MinIO
 - Runs in container paperless-minio



2. Layered Architecture (Backend)

- API Layer
 - Controllers: DocumentController
- Application / Business Layer
 - Services:
 - DocumentService
- Infrastructure / Data Access Layer
 - Repositories: DocumentRepository
 - DbContext: DataContext
- Integration Layer
 - Queue publisher: RabbitProducerService
 - Queue consumer: RabbitConsumerService

3. Class Diagram (Core Domain)



4. Sequence Diagram – Upload Document

Sequence for POST /api/documents/upload:

Actors: **User** → **Web UI** → **REST API** → **RabbitMQ** → **OCR Worker** → **MinIO** → **OCR Engine** → **REST API**

1. Frontend sends POST /api/documents with file/metadata.
2. API validates request.
3. API calls `DocumentService.CreateDocument(dto)`.
4. Service calls `DocumentRepository.Add(document)`.
5. Repository saves via `DbContext` to PostgreSQL.

6. API saves PDF in MinIO
7. Service / API publishes a message via DocumentQueuePublisher.
8. RabbitMQ receives message.
9. OCR worker receives message
10. OCR Worker receives document from MinIO
11. OCR worker generates the summary
12. RabbitMQ receives message with summary from OCR worker.
13. REST API receives message with generated summary
14. API calls Repo to update DB entry with summary

